

Enterprise Tool Integration: Error Recovery, Fallbacks, and Prompt Guardrails

1. Introduction

When an LLM agent is connected to enterprise systems, tools may interact with databases, REST APIs, payment systems, order systems, or internal services.

These systems can fail.

For example:

```
Agent
|
v
Order API
|
X
Service unavailable
```

An enterprise agent should not simply stop when this happens.

Three important techniques help make tool integration more robust:

1. **Error recovery** — handle failures without crashing the application.
2. **Fallback options** — use an alternative system when the preferred system fails.
3. **Prompt guardrails** — constrain the agent's behavior.

This example uses an **order processing system**.

2. The Order Processing Scenario

Assume an agent needs to answer:

```
Check order ORD1001 and tell me its current status.
```

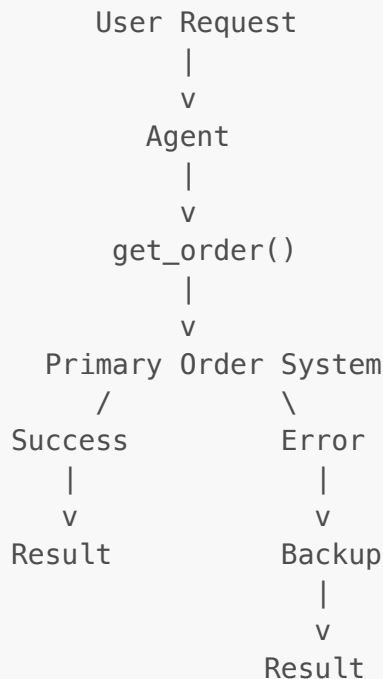
The application has two order systems:

```
Primary Order System
Backup Order System
```

Normally, the primary system should be used.

If it becomes unavailable, the backup system should be used.

The overall design is:



The important point is that **the fallback is handled by application code**, rather than asking the LLM to decide how infrastructure failures should be handled.

3. Defining the Primary Tool

The primary order system can be represented by a tool:

```

from langchain_core.tools import tool

@tool
def get_order_from_primary(order_id: str):
    """Get an order from the primary order system."""

    # Simulate a failure
    raise Exception("Primary order system is unavailable")
  
```

The exception simulates a real enterprise failure such as:

- REST API unavailable
- database connection failure
- network timeout
- service outage

In a real application, this function would make an actual API or database call.

4. Defining the Backup Tool

A second tool represents the backup system:

```
@tool
def get_order_from_backup(order_id: str):
    """Get an order from the backup order system."""

    orders = {
        "ORD1001": {
            "status": "SHIPPED",
            "customer": "John"
        }
    }

    return orders.get(order_id, "Order not found")
```

The backup system provides the same basic capability through a different implementation.

5. Implementing Error Recovery

Simply having a backup system is not enough.

The application needs to detect the failure.

A wrapper tool can handle this:

```
@tool
def get_order(order_id: str):
    """Get order details."""

    try:

        return get_order_from_primary.invoke({
            "order_id": order_id
        })

    except Exception as e:

        print("Primary system failed:", e)

        return get_order_from_backup.invoke({
```

```
        "order_id": order_id
    })
```

The `try` block attempts the primary operation.

The `except` block handles the failure.

Therefore, an exception in the primary system does not necessarily terminate the overall operation.

6. Implementing the Fallback

The fallback is implemented here:

```
except Exception as e:
    print("Primary system failed:", e)
    return get_order_from_backup.invoke({
        "order_id": order_id
    })
```

The sequence is deterministic:

```
get_order()
|
v
Try Primary
|
+----- Success ----> Return result
|
+----- Failure
|
v
Try Backup
|
v
Return result
```

The LLM does not need to know that the primary system failed.

It simply calls:

```
get_order("ORD1001")
```

The application handles the infrastructure details.

7. Why Put the Fallback Inside the Tool?

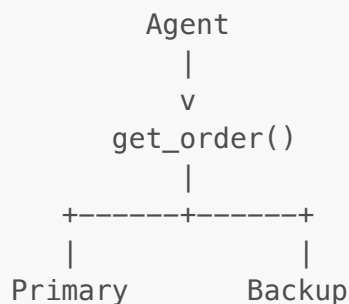
An alternative design would expose both systems to the agent:

```
get_order_from_primary()  
get_order_from_backup()
```

The LLM would then have to decide which one to call.

That creates unnecessary responsibility for the agent.

A better abstraction is:



The agent only needs to know:

```
"I need order information."
```

The application decides:

```
"Which enterprise system should provide it?"
```

This separation makes the tool interface simpler.

8. Adding an Order Cancellation Tool

Suppose the system also supports cancellation:

```
@tool  
def cancel_order(order_id: str):  
    """Cancel an order."""  
  
    return f"Order {order_id} has been cancelled."
```

Now the agent has two business capabilities:

```
tools = [  
    get_order,  
    cancel_order  
]
```

The agent can retrieve order information and perform cancellation when appropriate.

9. Prompt Guardrails

The agent should also be given behavioral rules.

For example:

```
SYSTEM_PROMPT = """"  
You are an enterprise order processing assistant.  
  
Follow these rules:  
  
1. Always use get_order() to retrieve order information.  
2. Never invent order information.  
3. Never cancel an order unless its current status is known.  
4. Do not cancel an order that is already DELIVERED.  
5. If order information cannot be retrieved, explain the problem.  
"""
```

These instructions are **prompt guardrails**.

They guide the agent's decisions.

For example, consider:

```
Cancel ORD1001.
```

The agent should not immediately call:

```
cancel_order("ORD1001")
```

Instead, it should first obtain the order status.

```
User
|
v
"Cancel ORD1001"
|
v
Agent
|
v
get_order()
|
v
Check status
|
v
Decide whether cancellation is appropriate
```

10. Prompt Guardrails Are Not Security Controls

Prompt guardrails are useful, but they should not be treated as hard security mechanisms.

For example:

```
Never cancel a delivered order.
```

is an instruction to the LLM.

It does not guarantee that the underlying application will reject an invalid cancellation.

For sensitive operations, the application should enforce the rule as well.

For example:

```
def cancel_order(order_id, status):

    if status == "DELIVERED":
        raise ValueError(
            "Delivered orders cannot be cancelled"
        )

    # Perform cancellation
```

Now there are two levels of protection:

```
LLM
|
| Prompt guardrail
v
Agent decision
|
| Application validation
v
Enterprise system
```

The application-level rule is the actual enforcement mechanism.

11. Key Takeaways

Enterprise tool integration requires more than simply connecting an LLM to an API.

A robust design separates responsibilities.

The agent handles reasoning:

```
"What should I do?"
```

The tool handles business capabilities:

```
"Get this order."
```

Application code handles reliability:

```
"What should happen if the API fails?"
```

Prompt guardrails guide the agent:

```
"What should the agent be allowed or expected to do?"
```

The resulting pattern is:

```
LLM Agent
|
| Business decision
v
```

```
Tool
|
| Application-controlled execution
v
Enterprise System
|
+----- Error ----> Recovery / Fallback
```

The central principle is:

Use the LLM for reasoning and decision-making, but keep reliability, security, and critical business-rule enforcement in application code.